EXP NO: 7	Image Captioning Using Cnn-Rnn Architecture

AIM:

To generate natural-language captions for images using an encoder-decoder architecture, with a pre-trained Vision Transformer (ViT) as the image encoder and a GPT-2 language model as the text decoder.

ALGORITHM:

1. Import the VisionEncoderDecoderModel, ViTImageProcessor and tokenizer from the Transformers library.
2. Load the pre-trained model nlpconnect/vit-gpt2-image-captioning (ViT encoder, GPT-2 decoder) and move it to the available device in evaluation mode.
3. Upload an image, open it and convert it to RGB.
4. Pre-process the image with the ViT image processor to obtain pixel values.
5. Generate the caption token ids with beam search (num_beams = 4, max_length = 30).
6. Decode the token ids into text and print the generated caption.

CODE:

```
# EXPERIMENT 7
# Image Captioning using CNN/Transformer Encoder and RNN-style Decoder

import torch
from transformers import VisionEncoderDecoderModel
from transformers import ViTImageProcessor
from transformers import AutoTokenizer
from PIL import Image
from google.colab import files

# Device
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Using device:", device)

# Load pretrained image captioning model

model_name = "nlpconnect/vit-gpt2-image-captioning" print("Loading

model...")

model = VisionEncoderDecoderModel.from_pretrained(model_name)

processor = ViTImageProcessor.from_pretrained(model_name) tokenizer

= AutoTokenizer.from_pretrained(model_name)

model = model.to(device)

model.eval()

print("Model loaded successfully!")

# Upload image

print("\nUpload an image:")

uploaded = files.upload()
```

```
image_path = list(uploaded.keys())[0]

print("Selected image:", image_path)

# Open image

image = Image.open(image_path).convert("RGB")

display(image)

# Generate caption
pixel_values = processor(
    images=image,
    return_tensors="pt"
).pixel_values

pixel_values = pixel_values.to(device)

with torch.no_grad():

    output_ids = model.generate(
        pixel_values,
        max_length=30,
        num_beams=4,
        early_stopping=True
    )

# Decode caption
caption = tokenizer.decode(
    output_ids[0],
    skip_special_tokens=True
)

print("\nGenerated Caption:")
print(caption)
```

OUTPUT:

```
Using device: cpu
Loading model...
config.json: 100%                                                 4.61k/4.61k [00:00<00:00, 52.8kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable h
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hu
pytorch_model.bin: reconstructing file: 100%                      982MB /  982MB, 67.3MB/s
pytorch_model.bin: downloading bytes:                             910MB, 54.0MB/s
model.safetensors: reconstructing file: 100%                      982MB /  982MB, 65.1MB/s
model.safetensors: downloading bytes:                              909MB, 58.2MB/s
Loading weights: 100%                                          445/445 [00:00<00:00, 4951.49it/s]
[transformers] VisionEncoderDecoderModel LOAD REPORT from: nlpconnect/vit-gpt2-image-captioning
Key                                                  | Status     | |
-----------------------------------------------------+------------+--+-
decoder.transformer.h.{0...11}.crossattention.masked_bias | UNEXPECTED | |
decoder.transformer.h.{0...11}.attn.masked_bias      | UNEXPECTED | |
Notes:
- UNEXPECTED:    can be ignored when loading from different task/architecture; not ok if you expect
preprocessor_config.json: 100%                                    228/228 [00:00<00:00, 2.04kB/s]
tokenizer_config.json: 100%                                     241/241 [00:00<00:00, 2.01kB/s]
tokenizer.json: 100%                                          1.36M/1.36M [00:00<00:00, 6.76MB/s]
special_tokens_map.json: 100%                                     120/120 [00:00<00:00, 1.71kB/s]
```

RESULT:

The pre-trained ViT-GPT2 image captioning model (nlpconnect/vit-gpt2-image-captioning) was loaded successfully, and the pipeline for generating a caption from an uploaded image using beam search was implemented.